

บทที่ 3

ระบบปฏิบัติการลินุกซ์และโพรเซส

3.1 ระบบปฏิบัติการลินุกซ์ (Linux)

3.1.1 ประวัติความเป็นมาของลินุกซ์

ลินุกซ์ถือกำเนิดขึ้นในประเทศฟินแลนด์เมื่อปี ค.ศ. 1980 โดยนายลินุส โทรวาลด์ส (Linus Trovalds) นักศึกษาภาควิชาวิทยาการคอมพิวเตอร์ ในมหาวิทยาลัยเฮลซิงกิ มีความคิดวาระบบปฏิบัติการยูนิกซ์ (UNIX) บนพีซีในขณะนั้นยังมีความสามารถไม่เพียงพอกับความต้องการของผู้ใช้ ดังนั้นจึงเริ่มสร้างระบบปฏิบัติการยูนิกซ์ของตนเองขึ้น

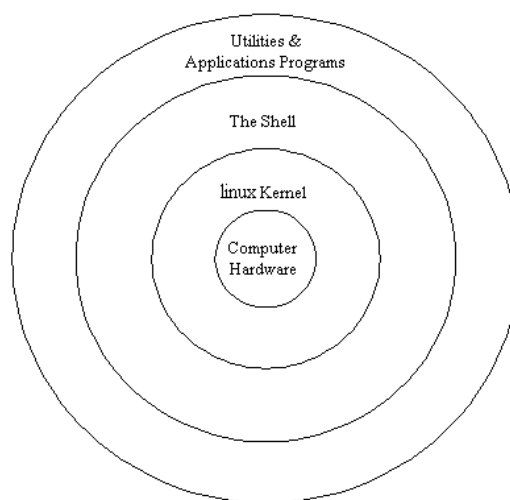
ลักษณะการพัฒนาลินุกซ์ได้ใช้อินเทอร์เน็ตช่วยโดยอาศัยความร่วมมือของนักพัฒนาจากสถานที่ต่างๆ จึงสังเกตได้ว่าการพัฒนาของระบบปฏิบัติการลินุกซ์เป็นไปอย่างรวดเร็ว

3.1.2 ลินุกซ์คืออะไร

ลินุกซ์เป็นระบบปฏิบัติการแบบ 32 บิตอย่างแท้จริงที่สนับสนุนการใช้งานแบบหลายงาน (Multitasking) และแบบหลายผู้ใช้ (Multi-user) โดยมีระบบการทำงานคล้ายระบบยูนิกซ์ (UNIX Clone) มีระบบเอ็กซ์วินโดว์ (X-windows) ซึ่งเป็นระบบการติดต่อกับผู้ใช้แบบกราฟิกที่ไม่ขึ้นระบบปฏิบัติการ (Operating system) หรือฮาร์ดแวร์ใดๆ และมีโพรโตคอล TCP/IP ซึ่งเริ่มแรกได้ถูกพัฒนาขึ้นมาเพื่อใช้งานกับระบบเน็ตเวิร์กของยูนิกซ์ที่ใช้งานกันอย่างกว้างขวางในการเชื่อมกันของเครือข่ายอินเทอร์เน็ต ระบบเน็ตเวิร์กบนลินุกซ์ถือได้ว่ามีความสมบูรณ์ต่อการใช้งานจริง เช่น สามารถที่จะเชื่อมต่อเป็นเครือข่ายผ่านระบบแลนทั่วไป หรือการเชื่อมต่อในระบบรีโมตเทอร์มินัลผ่านโมเด็ม

ลินุกซ์มีความเข้ากันได้ (Compatible) กับมาตรฐานโพสซิก (POSIX) ซึ่งเป็นมาตรฐานอินเทอร์เน็ตเฟสที่ระบบยูนิกซ์ส่วนใหญ่ต้องมี และมีรูปแบบบางส่วนที่คล้ายกับระบบปฏิบัติการยูนิกซ์จากค่ายเบิร์กลีย์ (Berkeley) และซิสเต็มวี (System V)

โดยเทคนิคแล้วลินุกซ์เป็นเพียงเคอร์เนล (Kernel) ของระบบปฏิบัติการ มีหน้าที่สำคัญ คือ การจัดการกับโพรเซสและการจัดการกับไฟล์ รวมทั้งการจัดการกับอุปกรณ์ต่างๆ เป็นส่วนที่จะทำหน้าที่ในการบริหารรวมทั้งควบคุมระบบทั้งหมด มีโครงสร้างการทำงาน ดังรูป



รูปที่ 3-1 โมเดลของระบบปฏิบัติการลินุกซ์

นอกจากนี้แล้วลินุกซ์ยังมีลักษณะพิเศษในการจัดการกับหน่วยความจำและไฟล์ คือไฟล์เอกซ์คิวต์เทเบิล (Executable file) สามารถแชร์พื้นที่ในหน่วยความจำได้ ลักษณะในการทำงานแบบนี้เรียกว่า copy on write page ซึ่งเป็นการเพิ่มประสิทธิภาพในการใช้งานหน่วยความจำ สามารถทำงานในระบบดีมานด์เพจิง (Demand paging) ซึ่งเป็นการนำเอาโปรแกรมที่ใช้งานบ่อยมาเก็บไว้ในหน่วยความจำ และถ้ามีการใช้งานระบบมากขึ้น ความจุสักระยะจะถูกปรับอัตโนมัติ เมื่อต้องการที่จะใช้หน่วยความจำเพิ่มขึ้น ลินุกซ์สามารถที่จะนำพื้นที่บนดิสก์มาทำหน้าที่เสมือนหน่วยความจำได้ (swap space) แต่ก็มีข้อเสียอันเนื่องมาจากการแอ็กเซสข้อมูลที่ช้าของดิสก์เมื่อเทียบกับหน่วยความจำจริง

นอกจากนี้แล้ว ถ้ามีแอปพลิเคชันใดใช้งานไลบรารีเหมือนกัน ลินุกซ์สามารถที่จะจัดการเพื่อให้แอปพลิเคชันเหล่านั้นใช้งานไลบรารีร่วมกัน เป็นการลดขนาดแอปพลิเคชันลง (Dynamic shared libraries)

นอกจากนี้ในปัจจุบัน ลินุกซ์ยังสามารถใช้งานบนแพลตฟอร์ม (Platform) ต่างๆ ได้ ตัวอย่างเช่น DEC Alpha, Motorola Power-PC เป็นต้น

3.1.3 สาเหตุที่เลือกใช้ลินุกซ์ในโครงการ

สาเหตุที่เลือกใช้ลินุกซ์ในโครงการนี้ คือ ลินุกซ์เป็นระบบปฏิบัติการที่แจ่มใส สามารถดาวน์โหลดจากอินเทอร์เน็ตโดยไม่ผิดกฎหมายและมีผู้นิยมใช้มาก มีผู้นำลินุกซ์ไปแก้ไขให้สามารถใช้งานได้บนตัวประมวลผลหลากหลาย นอกจากนี้ยังมีผู้พัฒนาแอปพลิเคชันสำหรับลินุกซ์ออกมาเรื่อยๆ

เนื่องจากลินุกซ์เป็นระบบปฏิบัติการ 32 บิตแบบแท่งจริง ทำให้ดึงความสามารถในการทำงานของคอมพิวเตอร์ออกมาได้เต็มกำลัง การทำงานของลินุกซ์จึงมีประสิทธิภาพและลินุกซ์มีคุณลักษณะของยูนิกซ์เต็มรูปแบบ ทำให้เป็นระบบหลายผู้ใช้และหลายงาน ซึ่งสนับสนุนโพรโทคอลแบบ TCP/IP, SLIP, PPP, UUCP และอื่นๆ โดยเวอร์ชันที่เลือกนำมาใช้เป็น Linux mandrake 8.0 และใช้เคอร์เนล 2.4.3

สำหรับการใช้ประโยชน์ของลินุกซ์ในด้านอื่นๆ สามารถใช้ลินุกซ์ในการศึกษาระบบปฏิบัติการยูนิกซ์ และแพ็คเกจของลินุกซ์ได้รวบรวมโปรแกรมที่ใช้จำลองสภาพการทำงานของดอส เพื่อให้โปรแกรมที่ทำงานบนดอส สามารถทำงานบนลินุกซ์ สำหรับวินโดวส์มีซอฟต์แวร์ที่สามารถทำให้ลินุกซ์รันแอปพลิเคชันของวินโดวส์บนระบบ X ชื่อ WABI (Windows Applications Binary Interface)

3.1.4 ระบบไดเรกทอรีบนลินุกซ์ (Linux Directory System)

ไดเรกทอรีเป็นกลุ่มของไฟล์ ข้อมูลภายในไดเรกทอรีจะประกอบด้วยไฟล์ หรือซับไดเรกทอรี ถ้ามีไดเรกทอรี A แล้วภายในไดเรกทอรี A มีไดเรกทอรีอื่นอยู่ด้วยชื่อไดเรกทอรี B จะเรียกไดเรกทอรี A ว่าเป็นแพเรนท์ไดเรกทอรี (Parent Directory) ส่วนไดเรกทอรี B จะเรียกว่าเป็นซับไดเรกทอรี

จำนวนไฟล์หรือซับไดเรกทอรีสำหรับลินุกซ์นั้นไม่ได้ไม่จำกัดขึ้นอยู่กับฮาร์ดดิสก์ว่ามีพื้นที่ในการเก็บข้อมูลมากเท่าใด แต่โดยทั่วไปแล้วโครงสร้างของไฟล์จะประกอบไปด้วย root (Root Directory) ซึ่งเป็นแพเรนท์ไดเรกทอรีทั้งหมดบนระบบ ซึ่งจะแสดงด้วยตัวอักษร “/” หมายถึงว่า ตอนนี้อยู่ที่ไดเรกทอรีสูงสุดแล้ว ภายใต้แพเรนท์ไดเรกทอรีจะประกอบไปด้วยไดเรกทอรีที่สำคัญอยู่จำนวนหนึ่งเป็นไดเรกทอรีหลักของระบบ

- `"/bin"` เก็บโปรแกรมที่สำคัญของระบบเช่น `"ls"` (ใช้แสดงรายชื่อไฟล์), `"cp"` (ใช้คัดลอกไฟล์ข้อมูล) โดยเก็บเป็นไบนารีไฟล์ (binary file) ซึ่งเป็นภาษาเครื่อง
- `"/boot"` เก็บไฟล์ที่เก็บส่วนสำคัญสำหรับการบูตระบบ
- `"/dev"` เป็นไฟล์ที่ใช้แทนอุปกรณ์ต่าง ๆ ของระบบ เช่น `"/dev/hda"` จะแทนฮาร์ดดิสก์, `"/dev/tty"` จะแทนเทอร์มินอล (terminal)
- `"/etc"` คำที่ใช้เซตการทำงานของโปรแกรมต่าง ๆ จะถูกเก็บลงในไดเรกทอรีนี้ เช่น `lilo.conf` , `syslog.conf` เป็นต้น
- `"/home"` เก็บข้อมูลต่าง ๆ ของผู้ใช้งานในระบบ เช่น ถ้าผู้ใช้ชื่อ `"mint"` คำปกติของไดเรกทอรีที่เก็บไฟล์จะเป็น `"/home/mint"` เป็นต้น
- `"/lib"` เก็บไฟล์ไลบรารี ของระบบ
- `"/proc"` เก็บข้อมูลเกี่ยวกับโปรเซสและเคอร์เนล
- `"/root"` เก็บข้อมูลของผู้ดูแลระบบ
- `"/sbin"` เก็บโปรแกรมที่ผู้ดูแลระบบใช้ในการดูแลรักษาระบบ เช่น `"shutdown"`, `"fsck"` เป็นต้น
- `"/usr"` เก็บโปรแกรมที่ติดตั้งโดยผู้ใช้งาน และนอกจากนั้นหากได้ติดตั้งส่วนของซอร์สโค้ด (source code) ของเคอร์เนลไว้ ก็จะสามารถดูได้ที่ `"/usr/src"`
- `"/var"` เก็บไฟล์ที่ใช้ดูแลระบบเช่น ล็อกไฟล์ (log file) ของระบบ และข้อมูลที่มีการเปลี่ยนแปลงอยู่บ่อยครั้ง เช่น อี-เมล เป็นต้น

3.1.5 การพัฒนาระบบงานบนลินุกซ์

ลินุกซ์ได้ทำการเตรียม เครื่องมือพัฒนาโปรแกรมให้เราไว้อย่างครบครันซึ่งจะมีตั้งแต่แอปพลิเคชันมาตรฐานคือ C/C++ คอมไพเลอร์ของ GNU และหากเราต้องการพัฒนาระบบบนเอ็กซ์ (X) ก็มี TCL/TK เตรียมไว้ให้ด้วย

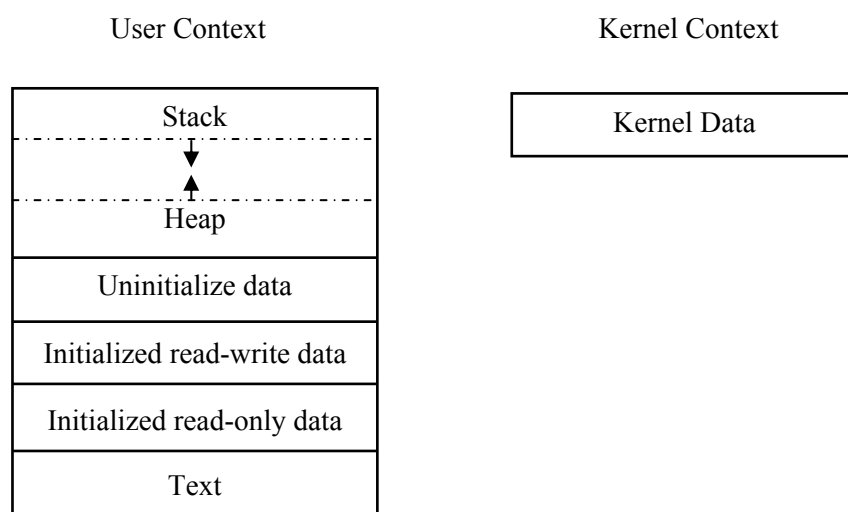
สำหรับคอมไพเลอร์ภาษาอื่น ๆ ก็มีเช่น เพิร์ล(Perl), สمولล์ทอล์ก(Smalltalk), พาสคาล(Pascal), ลิสป(Lisp) เป็นต้น ถ้าคุณมีความเชี่ยวชาญการเขียนโปรแกรมแบบเอ็กซ์เบส (X-Base) หรือ ฟ็อกซ์โปร(FoxPro) บนลินุกซ์ก็มีดาต้าเบสที่มีการเขียนโปรแกรมแบบนี้ให้เช่นกัน และล่าสุดลินุกซ์ก็มี จาวาคอมไพเลอร์ให้สำหรับผู้ที่ชอบการเขียนแอปเพลตจาวา สำหรับรันบนอินเทอร์เน็ตด้วย

3.2 โพรเซสและข้อมูลเกี่ยวกับโพรเซส

3.2.1 ความหมายของโพรเซส

โพรเซส หมายถึง ชุดคำสั่งในหน่วยความจำที่กำลังทำงานอยู่ในขณะนั้น โดยมีชุดควบคุมการทำงาน 1 ชุดเป็นของตัวเองซึ่งได้ใช้ทรัพยากรของระบบไปส่วนหนึ่ง ทั้งนี้ต้องทำความเข้าใจไว้ก่อนว่า โพรเซสกับโปรแกรมไม่ใช่สิ่งเดียวกันยกตัวอย่างเช่น ในระบบมัลติยูสเซอร์สมมติว่ามีผู้ใช้งาน 50 คน กำลังใช้โปรแกรม pine เพื่อทำการอ่านอีเมลล์พร้อมๆ กันก็จะมีโพรเซสเกิดขึ้น 50 โพรเซสที่กำลังใช้โปรแกรม pine เพียงโปรแกรมเดียว

3.2.2 โครงสร้างของโพรเซส



รูปที่ 3-2 แสดงโครงสร้างอย่างคร่าว ๆ ของโพรเซส

จากรูปเราแบ่งโครงสร้างโพรเซส 2 ส่วน คือ

ส่วน User context

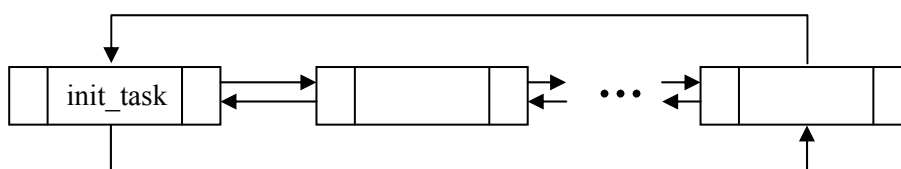
ประกอบไปด้วยส่วนต่าง ๆ ดังนี้

- text เก็บภาษาเครื่อง (machine instruction) ที่ใช้รัน โดยปกติถูกกำหนดให้มีเพอร์มิสชัน (permission) เป็น read only ดังนั้นโปรเซสจึงไม่สามารถแก้ไขข้อมูลส่วนนี้ได้
- data เก็บข้อมูลที่ใช้ในโปรแกรม แบ่งเป็น 3 ส่วนย่อยคือ uninitialized data, initialized read-write data และ initialized read only data
- heap ใช้งานในขณะรันโปรเซส เพื่อกำหนดพื้นที่หน่วยความจำให้เพียงพอต่อความต้องการในการใช้งาน
- stack สำหรับควบคุมการใช้งานตัวแปร การเรียกใช้ฟังก์ชันและการส่งค่ากลับ

ส่วน Kernel context

ถูกควบคุมและใช้งานโดยเคอร์เนลเท่านั้น เพื่อเก็บข้อมูลที่จำเป็นสำหรับควบคุมและติดตามการทำงานของโปรเซสซึ่งการที่เคอร์เนลจะจัดการกับโปรเซสได้นั้น ตัวเคอร์เนลเองต้องทราบข้อมูลต่างๆ ที่เกี่ยวกับโปรเซสแต่ละตัวเก็บไว้เช่น มีหมายเลขประจำโปรเซส (Process ID) มีค่าเท่าไร สถานะการทำงานของโปรเซสกำลังประมวลผลอยู่หรือถูกบล็อก (block) อยู่หรือไม่ มีแอดเดรสสเปซ (address space) อยู่ที่ตำแหน่งใด มีไพโอริตี้(priority)เท่าไร เป็นต้น ซึ่งข้อมูลต่าง ๆ เหล่านี้ แต่ละโปรเซสมีเก็บไว้ในแต่ละโปรเซสในส่วนที่เรียกว่า process descriptor โดยรายละเอียดโครงสร้างที่เป็นส่วนประกาศในภาษาC สามารถดูได้ที่ `/usr/include/linux/sched.h` ซึ่งอาจจะแตกต่างกันไปตามเวอร์ชันของเคอร์เนล

ในโครงสร้างข้อมูลนี้ มีโครงสร้างข้อมูลที่ใช้ชี้ไปยัง Process descriptor ถัดไปและก่อนหน้า ซึ่งหากลองพิจารณาจะเห็นว่าโครงสร้างเป็น circular doubly link list ดังรูปที่ 3-3 ซึ่งส่วนหัวของลิงค์ลิสต์นี้เป็น `init_task` descriptor ซึ่งเป็นโปรเซสแรกของลินุกซ์ที่เกิดขึ้นในช่วงการบูทระบบซึ่งมักถูกเรียกว่า process 0 หรือ swapper จะมีหน้าที่ในการบริหารงานโปรเซส



รูปที่ 3-3 ลักษณะของการเชื่อมโยงกันของ process descriptor

ในการจัดการกับโปรเซสทั้งหมดนั้นเคอร์เนลมีอาร์เรย์ ที่เรียกว่า Task array โดยสมาชิกใน array นี้ประกอบด้วย pointer ชี้ไปยัง process descriptor ของโปรเซสต่างๆ มีไว้เพื่อความสะดวกของการจัดการโปรเซสของเคอร์เนลเอง

3.2.3 การตรวจสอบข้อมูลของโพรเซส

มีคำสั่งอยู่หลายคำสั่งในลินุกซ์ที่ใช้ในการตรวจสอบข้อมูลของโพรเซส เช่น

- คำสั่ง **ps**

แสดงข้อมูลของโพรเซสที่อยู่ในไครเรกทอรี /proc (ซึ่งจะขออธิบายส่วนนี้ในหัวข้อถัดไป)

```

multimedia31 - SecureCRT
File Edit View Options Transfer Script Window Help

[root@multimedia31 root]# ps -aux | more
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1  0.0  0.9  1264   432 ?        S    Mar15   0:04 init [3]
root             2  0.0  0.0      0     0 ?        SW   Mar15   0:00 [keventd]
root             3  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kapmd]
root             4  0.0  0.0      0     0 ?        SWN  Mar15   0:00 [ksoftirqd_CPU0]
root             5  0.0  0.0      0     0 ?        SW   Mar15   0:01 [kswapd]
root             6  0.0  0.0      0     0 ?        SW   Mar15   0:00 [bdflush]
root             7  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kupdated]
root             8  0.0  0.0      0     0 ?        SW   Mar15   0:00 [mdrecoveryd]
root            12  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kjournald]
root            68  0.0  0.0      0     0 ?        SW   Mar15   0:00 [khubd]
root           161  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kjournald]
root           162  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kjournald]
root           163  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kjournald]
root           164  0.0  0.0      0     0 ?        SW   Mar15   0:00 [kjournald]
root           422  0.0  0.0      0     0 ?        SW   Mar15   0:00 [eth0]
root           476  0.0  1.1  1324   528 ?        S    Mar15   0:00 syslogd -m 0
root           480  0.0  0.9  1264   416 ?        S    Mar15   0:00 klogd -x
rpc            497  0.0  0.9  1404   412 ?        S    Mar15   0:00 portmap
rpcuser       517  0.0  1.2  1444   584 ?        S    Mar15   0:00 rpc.statd
root          582  0.0  0.9  1256   424 ?        S    Mar15   0:00 /usr/sbin/apmd -p
root          620  0.0  2.5  3200  1148 ?        S    Mar15   0:02 /usr/sbin/sshd
root          634  0.0  1.7  1996   808 ?        S    Mar15   0:00 xinetd -stayalive
root          651  0.0  2.2  3932  1012 ?        S    Mar15   0:00 /bin/sh /usr/bin/
mysql         683  0.0  2.8 28624  1296 ?        S    Mar15   0:00 /usr/libexec/mysq
--More--
Ready                               ssh1: 3DES   27, 9   27 Rows, 81 Cols  VT100   NUM

```

รูปที่ 3-4 แสดงการผลลัพธ์จากการรันคำสั่ง **ps**

- คำสั่ง **top**

แสดงข้อมูลของโพรเซสเช่นกัน แต่มีการสแกนข้อมูลอยู่ตลอดเวลา

```

multimedia31 - SecureCRT
File Edit View Options Transfer Script Window Help

4:03pm up 17:56, 1 user, load average: 0.00, 0.00, 0.00
53 processes: 51 sleeping, 2 running, 0 zombie, 0 stopped
CPU states: 0.5% user, 0.7% system, 0.0% nice, 98.6% idle
Mem: 45644K av, 42204K used, 3440K free, OK shrd, 3940K buff
Swap: 192740K av, 6908K used, 185832K free

  PID USER      PRI  NI  SIZE  RSS SHARE STAT   %CPU %MEM   TIME COMMAND
 5206 root        15   0  1012  1012   820 R    1.3  2.2   0:00 top
    1 root        15   0   456   432   404 S    0.0  0.9   0:04 init
    2 root        15   0     0     0     0 SW    0.0  0.0   0:00 keventd
    3 root        15   0     0     0     0 SW    0.0  0.0   0:00 kapid
    4 root        34  19     0     0     0 SWN   0.0  0.0   0:00 ksoftirqd_CPU0
    5 root        15   0     0     0     0 SW    0.0  0.0   0:01 kswapd
    6 root        15   0     0     0     0 SW    0.0  0.0   0:00 bdflush
    7 root        15   0     0     0     0 SW    0.0  0.0   0:00 kupdated
    8 root        25   0     0     0     0 SW    0.0  0.0   0:00 mdrecoveryd
   12 root        15   0     0     0     0 SW    0.0  0.0   0:00 kjournald
   68 root        16   0     0     0     0 SW    0.0  0.0   0:00 khubd
  161 root        15   0     0     0     0 SW    0.0  0.0   0:00 kjournald
  162 root        15   0     0     0     0 SW    0.0  0.0   0:00 kjournald
  163 root        15   0     0     0     0 SW    0.0  0.0   0:00 kjournald
  164 root        15   0     0     0     0 SW    0.0  0.0   0:00 kjournald
  422 root        15   0     0     0     0 SW    0.0  0.0   0:00 eth0
  476 root        15   0   536   528   488 S    0.0  1.1   0:00 syslogd
  480 root        15   0   432   416   416 S    0.0  0.9   0:00 klogd
  497 rpc          15   0   484   412   412 S    0.0  0.9   0:00 portmap
  517 rpcuser     16   0   672   584   584 S    0.0  1.2   0:00 rpc.statd
Ready                               ssh1: 3DES   6, 1   27 Rows, 81 Cols  VT100   NUM

```

รูปที่ 3-5 แสดงผลลัพธ์ของการรันคำสั่ง **top**

3.2.4 การจัดเก็บข้อมูลใน /proc

ภายในไดเรกทอรี /proc เป็นที่เก็บข้อมูลรายละเอียดเกี่ยวกับเคอร์เนลและโพรเซสซึ่งในที่นี้ขอกล่าวถึงเฉพาะในส่วนที่เกี่ยวกับโพรเซส

/proc/[number]

เก็บข้อมูลเกี่ยวกับโพรเซสตามหมายเลขที่กำลังรันไว้ เช่น /proc/1243 ภายในไดเรกทอรีนี้เก็บข้อมูลเกี่ยวกับโพรเซสที่มีหมายเลขโพรเซสเป็น 1243 ไว้ ซึ่งภายในมีรายละเอียดดังนี้

- 1) **ไฟล์ cmdline** : เก็บคำสั่งที่โพรเซสนั้นประมวลผล
- 2) **ไฟล์ environ** : เก็บค่าตัวแปรสภาพแวดล้อมของโพรเซสนั้น ๆ
- 3) **ไดเรกทอรี fd** : เก็บ Symbolic link ของ file descriptor ของไฟล์ที่โพรเซสนั้นได้เรียกใช้งาน
- 4) **cwd Symbolic link** : เป็นลิงก์ที่ชี้ไปยังไดเรกทอรีที่โพรเซสทำงานอยู่ (Current working directory)
- 5) **exe Symbolic link** : เป็นลิงก์ที่ชี้ไปยังไฟล์ที่โพรเซสกำลังประมวลผลอยู่
- 6) **ไฟล์ maps** : เก็บส่วน Memory map ของโพรเซส ซึ่งประกอบด้วยช่วงตำแหน่ง, permission, offset เป็นต้น
- 7) **root Symbolic link** : เป็นลิงก์ไปยังรูตไดเรกทอรี (root directory)
- 8) **ไฟล์ statm** : เก็บข้อมูลการใช้งานหน่วยความจำของโพรเซส
- 9) **ไฟล์ stat** : เก็บข้อมูลรายละเอียดเกี่ยวกับโพรเซส ซึ่งไฟล์นี้เป็นไฟล์ที่มีรายละเอียดมากที่สุดโดยมีการเก็บข้อมูลเรียง ๆ กันซึ่งลำดับการเรียงข้อมูลนั้นอาจแตกต่างกันไปตามเวอร์ชันของเคอร์เนล ซึ่งความหมายของแต่ละตัวแปรดูได้ตามตารางที่ 2-1

ตัวแปร	ความหมาย
pid	หมายเลขประจำโพรเซส
cmd	คำสั่งที่โพรเซสทำการประมวลผล
state	สถานะของโพรเซส
ppid	หมายเลขประจำโพรเซสของโพรเซสที่เป็นผู้สร้างโพรเซสนี้
pgrp	หมายเลขกลุ่มของโพรเซส
session	หมายเลข session ของโพรเซส
tty	tty ที่โพรเซสใช้
tpgid	หมายเลขประจำโพรเซสของ tty ที่ใช้
flags	flag ของโพรเซส
minflt	ค่า minor page fault
cminflt	ค่า minor page fault ที่โดยรวมค่าที่โพรเซสถูกของโพรเซสนี้ด้วย
majflt	ค่า major page fault
cmajflt	ค่า major page fault ที่โดยรวมค่าที่โพรเซสถูกของโพรเซสนี้ด้วย

utime	User time ในหน่วย jiffies (1/100 วินาที)
stime	System time ในหน่วย jiffies
cutime	User time โดยรวมเวลาของโปรเซสลูกด้วย ในหน่วย jiffies
cstime	System time โดยรวมเวลาของโปรเซสลูกด้วย ในหน่วย jiffies
priority	ค่า static priority ของโปรเซส
nice	เป็นค่าไพโรอริตี้ที่สามารถเปลี่ยนแปลงได้ (Dynamic priority)
start_time	เวลาที่โปรเซสเริ่มทำงาน
vsize	ขนาดของ Visual Memory ในหน่วย byte
rss	Resident set size
rlim	ค่าจำกัดของ RSS
start_code	จุดเริ่มต้นของโค้ดเซกเมนต์ (code segment)
end_cod	จุดสิ้นสุดต้นของโค้ดเซกเมนต์
start_stack	จุดเริ่มต้นของสแต็กเซกเมนต์ (stack segment)
esp	ตำแหน่งของสแต็กเฟรมปัจจุบัน (current stack frame)
eip	ตำแหน่งของสแต็กเฟรมปัจจุบัน (current stack frame)
pending.signal	เก็บสัญญาณที่โปรเซสต้องการตรวจสอบว่ายังค้างอยู่ในระบบหรือไม่
blocked.signal	เก็บสัญญาณที่โปรเซสต้องการบล็อก
Sigign	เก็บสัญญาณที่โปรเซสไม่สนใจ
Sigcatch	เก็บสัญญาณที่โปรเซสกำลังรอ

ตารางที่ 3-1 แสดงความหมายของข้อมูลในไฟล์ `/proc/[number]/stat`

- 10) **ไฟล์ status** : เก็บข้อมูลของโปรเซสเช่นเดียวกันโดยแสดงให้อยู่ในรูปแบบที่เข้าใจได้ง่าย แต่ข้อมูลน้อยกว่าในไฟล์ stat

3.2.5 ล็อกไฟล์ acct

ไฟล์นี้มีหน้าที่เก็บข้อมูลของโปรเซสที่ผู้ใช้ได้เรียกใช้งานซึ่งถูกบันทึกเมื่อโปรเซสจบการทำงานปกติแล้วจะเก็บไฟล์นี้ไว้ที่ `/var/account` โดยจะเก็บข้อมูลเป็นรายวันแล้วมีการตัดข้อมูลที่ผ่านมาแบ็กอัพไว้ซึ่งทั้งหมดนี้ทำโดยใช้ `logrotate` ลักษณะโครงสร้างจะสามารถเข้าไปดูได้ที่ `/usr/include/linux/acct.h` ซึ่งมีลักษณะดังนี้

ตัวแปร	คำอธิบาย
ac_flag	Flags ของโปรเซส
ac_uid	หมายเลขประจำตัวของผู้ที่เป็นเจ้าของโปรเซส
ac_gid	หมายเลขประจำกลุ่มของผู้ที่เป็นเจ้าของโปรเซส

ac_tty	เทอร์มินอลที่ใช้ในการรันโปรแกรม
ac_btime	เวลาที่โปรแกรมเกิดขึ้น
ac_untime	User time
ac_stime	System Time
ac_etime	Elapsed Time
ac_mem	ปริมาณการใช้หน่วยความจำโดยเฉลี่ย
ac_io	ปริมาณการใช้ i/o
ac_rw	จำนวนบล็อกที่เขียนหรืออ่าน
ac_minflt	ค่า Minor Pagefaults
ac_majflt	ค่า Major Pagefaults
ac_swaps	จำนวน swap ที่ใช้
ac_exitcode	Exit code ของโปรแกรม
ac_comm[ACCT_COMM + 1]	คำสั่งที่ใช้ในโปรเซส

ตารางที่ 3-2 แสดงความหมายของตัวแปรในโครงสร้าง *struct acct*

คำสั่งที่ใช้ตรวจสอบข้อมูลในไฟล์นี้ก็มีเช่น lastcomm, sa เป็นต้น

คำสั่ง lastcomm

เป็นคำสั่งที่ใช้แสดงข้อมูลการใช้งานคำสั่งหรือโปรแกรมต่างๆที่ผ่านมา โดยจะดึงค่าจากไฟล์ pacct มาใช้งาน ถ้าไม่มีการระบบออพชั่นใดๆ จะแสดงข้อมูลทั้งหมดที่มีอยู่ในไฟล์ pacct ออกมา แต่เราสามารถระบุ ชื่อผู้ใช้งาน, คำสั่ง หรือชนิดของเทอร์มินอลที่เรียกใช้ตามหลังได้ด้วย เช่น

```
lastcomm a.out tty0
```

รายละเอียดของข้อมูลที่แสดงมีดังนี้

- คำสั่งหรือโปรแกรมที่ถูกเรียกใช้
- แพลก
 - S : คำสั่งนั้นถูกเรียกใช้โดย super-user
 - F : คำสั่งนั้นมีการ fork แต่ไม่มีการเรียกใช้งาน
 - C : คำสั่งนั้นทำงานในโหมดที่เข้ากันได้กับ PDP-11
 - D : คำสั่งมีการสร้างไฟล์ core เมื่อจบการทำงาน
 - X : คำสั่งส่งสัญญาณ SIGTERM เมื่อจบการทำงาน
- ชื่อผู้ที่เรียกใช้งาน
- เวลาที่จบการทำงาน

```

multimedia31 - SecureCRT
File Edit View Options Transfer Script Window Help

[root@multimedia31 root]# lastcomm lmore
pico          root      stdin    0.03 secs Sun Mar 16 16:05
pine          S      root      stdin    0.17 secs Sun Mar 16 16:05
ls            root      stdin    0.02 secs Sun Mar 16 16:05
vim           root      stdin    0.15 secs Sun Mar 16 16:05
bash          F      root      stdin    0.01 secs Sun Mar 16 16:05
bash          F      root      stdin    0.00 secs Sun Mar 16 16:05
vim           root      stdin    0.17 secs Sun Mar 16 16:05
ls            root      stdin    0.01 secs Sun Mar 16 16:05
ls            root      stdin    0.02 secs Sun Mar 16 16:05
more          root      stdin    0.02 secs Sun Mar 16 16:05
ps            root      stdin    0.12 secs Sun Mar 16 16:05
lastcomm      root      stdin    1.12 secs Sun Mar 16 16:04
lastcomm      root      stdin    0.23 secs Sun Mar 16 16:04
lastcomm      root      stdin    0.23 secs Sun Mar 16 16:04
lastcomm      root      stdin    0.23 secs Sun Mar 16 16:04
lastcomm      root      stdin    0.23 secs Sun Mar 16 16:04
lastcomm      X      root      stdin    0.47 secs Sun Mar 16 16:03
more          root      stdin    0.02 secs Sun Mar 16 16:03
top           root      stdin    1.07 secs Sun Mar 16 16:02
top           root      stdin    0.55 secs Sun Mar 16 16:01
ls            root      stdin    0.02 secs Sun Mar 16 16:01
ls            root      stdin    0.02 secs Sun Mar 16 16:01
ls            root      stdin    0.02 secs Sun Mar 16 16:01
stty          root      stdin    0.01 secs Sun Mar 16 16:01
bash          F      root      stdin    0.00 secs Sun Mar 16 16:01
--More--
Ready          ssh1: 3DES  27, 9  27 Rows, 81 Cols  VT100  NUM

```

รูปที่ 3-6 แสดงสถิติของผู้ใช้งานโดยคำสั่ง `lastcomm`

3.3 ครอนและครอนแท็บ (Cron,Crontab)

ครอนเป็นระบบจัดการเอ็กซิกิวต์คำสั่งตามเวลาที่กำหนดไว้ล่วงหน้าโดยตัวครอนเดมอน (crond) ถูกรันจากไฟล์ `/etc/rc` หรือ `/etc/rc.local` โดยตัวเดมอนนี้เมื่อเริ่มต้นทำงานจะอ่านค่าเริ่มต้นจากไฟล์ `/var/spool/cron` ซึ่งเป็นส่วนที่ผู้ใช้แต่ละคนสามารถสร้างตารางเวลาของแต่ละคนได้ ชื่อไฟล์มีชื่อเดียวกันกับชื่อผู้ใช้ ส่วนอีกไฟล์หนึ่งเป็นการกำหนดตารางเวลาของระบบคือไฟล์ `/etc/crontab` โดยผู้ที่จัดการกับไฟล์นี้ได้ต้องเป็น root เท่านั้น

ครอนแท็บ เป็นไฟล์ที่ใช้กำหนดเวลาล่วงหน้าของระบบ อยู่ใน `/etc/crontab` คำสั่งที่รันจากครอนของระบบจะมีสิทธิ์ของ root ไฟล์ครอนแท็บมีข้อกำหนดการเขียนคือ บรรทัดที่ว่างและบรรทัดที่ขึ้นต้นด้วย # ถือว่าเป็นคอมเมนต์ ในไฟล์ครอนแท็บสามารถกำหนดค่าตัวแปรในรูปแบบดังนี้

name = value

รูปแบบการกำหนดตารางการทำงาน

ฟิลด์	minute	hour	day_of_month	month	day_of_week	[user]	command
ช่วงเวลาที่ใช้	0-59	0-23	0-31	0-12	0-7		

ถ้าค่าในฟิลด์เวลาเป็น * หมายถึงทุกค่าที่เป็นไปได้ นอกจากนี้ยังสามารถกำหนดค่าเป็นช่วง โดยใช้เครื่องหมาย - ได้ เช่น 8-11 ก็หมายถึงเวลา 8,9,10,11 หรือกำหนดเป็นลิสต์ได้เช่น 1,2,5,9 หรือ 1-4,8-12 ก็ได้

นอกจากนี้สามารถกำหนดเป็นสตีปได้ เช่น สำหรับ hour กำหนดเป็น 0-23/2 หมายถึง 0,2,4,6,8,10,12,14,16,18,20,22 ซึ่งหมายถึงให้ทำคำสั่งนั้นทุก 2 ชั่วโมง อาจเขียนง่ายๆ ได้ว่า */2 และ เครื่องหมาย % มีความหมายเท่ากับเครื่องหมายขึ้นบรรทัดใหม่ (new line)

สำหรับฟิลด์ month และ day of week สามารถใช้ชื่อเป็น ภาษาอังกฤษสามตัวแรก เช่น อาทิตย์ ก็เป็น sun เดือนธันวาคมก็จะเป็น Dec โดยไม่สนใจว่าจะเป็นตัวพิมพ์เล็กหรือพิมพ์ใหญ่

ถ้าใช้เป็นตัวเลขในฟิลด์ day of week เลขแต่ละตัวมีความหมายดังนี้

เลข 0,7 หมายถึง วันอาทิตย์

เลข 1 หมายถึง วันจันทร์

เลข 2 หมายถึง วันอังคาร

เลข 3 หมายถึง วันพุธ

เลข 4 หมายถึง วันพฤหัสบดี

เลข 5 หมายถึง วันศุกร์

เลข 6 หมายถึง วันเสาร์

ตัวอย่าง

30	4	1,15	*	5	- หมายถึงคำสั่งรันที่เวลา 4:30 ทุกวันที่ 1 และ 15 ของทุกเดือนรวมถึงวันศุกร์ด้วย
*	*	*	*	*	- หมายถึงคำสั่งรันทุกหนี่งนาทึของทุกวัน
23	0-23/2	*	*	*	- รันที่เวลา 23 นาทึหลังเที่ยงคืนแล้วรันทุกๆ 2 ชั่วโมงทุกวัน
15	4	*	*	sun	- รันทุกวันอาทิตย์เวลา 4:15

3.4 ภาษา C และภาษา C++

3.4.1 ภาษา C

ภาษา C เป็นภาษาเขียนโปรแกรมที่ถูกนำไปใช้อย่างกว้างขวาง โดยภาษา C เริ่มมีมาตั้งแต่ช่วงแรกๆ ที่ระบบปฏิบัติการยูนิกซ์ (UNIX) ถือกำเนิดขึ้นมา ภาษา C ได้ถูกคิดค้นขึ้นโดยนายเดนนิส ริตชี (Denise Ritchie) ที่ห้องแล็บเบล โดยพัฒนาปรับปรุงมาจากภาษา B ภาษา C สามารถติดต่อในระดับฮาร์ดแวร์ได้ดีกว่าภาษาระดับสูงเช่น เบสิก (Basic) ฟอรัทแรน (Fortran) ขณะเดียวกันก็มีคุณสมบัติของภาษาระดับสูงอยู่ด้วย ดังนั้นจึงได้จัดให้ภาษา C เป็นภาษาระดับกลาง สำหรับที่ภาษา C ได้รับการยอมรับอย่างกว้างขวางนั้นก็เนื่องมาจากเหตุผลดังต่อไปนี้

- ภาษา C เป็นภาษาที่มีการกำหนดมาตรฐานสำหรับทุกแพลตฟอร์ม ทำให้โครงสร้างทางภาษา, ไลบรารี (library) และฟังก์ชันต่างๆสามารถนำไปใช้ข้ามแพลตฟอร์มได้
- โปรแกรมที่เขียนด้วยภาษา C ทำงานได้เร็ว

- ภาษา C เป็นภาษาระบบของยูนิกซ์ทุกเวอร์ชัน

ในช่วงปี 1980 ได้มีการกำหนดมาตรฐานของภาษา C ให้มีความเป็นสากล โดยมาตรฐานที่กำหนดขึ้นมานี้เรียกว่ามาตรฐาน ANSI C ซึ่งเป็นมาตรฐานที่ผู้ผลิตคอมพิวเตอร์ภาษา C จะต้องอ้างอิง ทำให้ภาษา C สามารถใช้งานข้ามแพลตฟอร์มได้

3.4.2 ภาษา C++

C++ (ซีพลัสพลัส) เป็นภาษาระดับสูง (High-level language) ซึ่งกำลังได้รับความนิยมอยู่ในปัจจุบัน เนื่องจาก C++ ได้รวบรวมเทคนิคต่างๆ ซึ่งมีการคิดค้นและสั่งสมต่อเนื่องกันมาในช่วงเวลาหลายปี ดังนั้น C++ จึงเป็นภาษาที่มีประสิทธิภาพสูงสุดภาษาหนึ่งสำหรับารเขียนโปรแกรม

ผู้สร้างภาษา C++ คือ บจาร์น สโตรสตรัป (Bjarne Stroustrup) ซึ่งทำงานอยู่ห้องปฏิบัติการเบล (Bell Laboratories) ประเทศสหรัฐอเมริกา ภาษานี้สร้างขึ้นประมาณปี ค.ศ. 1980

C++ ประกอบด้วยทุกสิ่งที่มีในภาษา C และส่วนที่เพิ่มเติมเข้ามา ดังนั้นสามารถเขียนโปรแกรมภาษา C ใน C++ ได้แต่ทำกลับกันคือเขียนภาษา C++ ใน C ไม่ได้ เป็นสาเหตุให้ผู้นิยมใช้ C++ มากขึ้นเป็นลำดับ ส่วนสำคัญที่เพิ่มเติมจากภาษา C ได้แก่ การเขียนโปรแกรมแบบออบเจกต์ (Object) หรือ OOP (Object-Oriented Programming) ซึ่งเป็นวิธีการเขียนโปรแกรมที่สามารถใช้งานได้กว้างขวางนิยมใช้ในการเขียนโปรแกรมขนาดใหญ่ที่มีการทำงานสลับซับซ้อน เพราะสะดวกและง่ายแก่การเขียน การตรวจสอบ การปรับปรุง และการนำมาใช้ใหม่

C++ ได้รับมาตรฐาน 4 มาตรฐาน คือ ANSI (The American National Standards Institute - สถาบันมาตรฐานแห่งชาติอเมริกัน) มาตรฐาน ISO (International Organization of Standardization - องค์การระหว่างประเทศเกี่ยวกับมาตรฐาน) มาตรฐาน BSI (The British Standards Institute - สถาบันมาตรฐานแห่งอังกฤษ) มาตรฐาน DIN (The German National Standards Organization - องค์การมาตรฐานแห่งชาติเยอรมัน) การกำหนดมาตรฐานมีส่วนดีทำให้โปรแกรมที่เขียนขึ้นสามารถนำไปใช้ในระบบคอมพิวเตอร์ที่แตกต่างกันได้ทันทีโดยไม่ต้องปรับปรุงเปลี่ยนแปลงส่วนใดๆ การกำหนดมาตรฐานอาจเป็นเครื่องบ่งชี้ให้ทราบว่า C++ เป็นภาษาที่ได้รับความนิยมมากภาษาหนึ่ง

3.5 เชลล์สคริปต์ (Shell Script)

ในการติดต่อกับระบบปฏิบัติการลินุกซ์มักติดต่อผ่านโปรแกรมเล็กๆ โปรแกรมหนึ่ง เรียกว่า เชลล์ (Shell) เชลล์เป็นชั้นของแอปพลิเคชันที่ตีความคำสั่งจากผู้ใช้และส่งต่อไปให้กับเคอร์เนลของระบบ จากนั้นแสดงผลที่ได้จากเคอร์เนลกลับมาให้ผู้ใช้อีกที และจากการที่ส่วนใหญ่ผู้ใช้ต้องติดต่อกับระบบปฏิบัติการลินุกซ์ผ่านทางเชลล์นี้เองจึงทำให้คนส่วนใหญ่เข้าใจว่าเชลล์ที่เห็นนั้นคือระบบปฏิบัติการ ซึ่งอันที่จริงแล้วเชลล์เป็นเพียงแอปพลิเคชันบนระบบเท่านั้น

3.5.1 ประวัติของเชลล์

ในระบบยูนิกซ์รวมถึงลินุกซ์นั้น สามารถเลือกใช้เชลล์ได้หลายแบบ ซึ่งแตกต่างจากระบบดอสที่จำกัดให้ใช้เชลล์ที่ติดตั้งมาแล้วกับดอส คือ command.com ได้เพียงอย่างเดียว สำหรับโปรแกรมเชลล์ตัวแรกที่มีใช้กันนั้นคือ เบอร์นเชลล์ (Bourne Shell) ซึ่งตั้งชื่อตามชื่อผู้คิดค้นโปรแกรมนี้ขึ้น คือนาย สตีเฟน เบอร์น (Steven Bourne) เบอร์นเชลล์นี้แจกจ่ายไปพร้อมกับยูนิกซ์เวอร์ชัน 7 ซึ่งเป็นเวอร์ชันที่ได้รับความนิยมมากที่สุดตัวหนึ่งในปี ค.ศ.1979 ผู้ใช้ส่วนใหญ่รู้จักเบอร์นเชลล์จากการเรียกคำสั่งในระบบว่า sh ถึงแม้ว่าในปัจจุบันมีโปรแกรมเชลล์ออกมามากมาย เบอร์นเชลล์ก็ยังถือว่าเป็นโปรแกรมสำคัญพื้นฐานที่ยูนิกซ์ทุกระบบต้องมี นอกจากนี้โปรแกรมเชลล์ตัวอื่นๆ ที่ถูกพัฒนาขึ้นมาภายหลังส่วนใหญ่ก็มีความเข้ากันได้กับเบอร์นเชลล์อยู่ทั้งสิ้น

โปรแกรมเชลล์ที่พัฒนาขึ้นมาภายหลังและได้รับความนิยมอย่างสูงอีกตัวหนึ่งก็คือ ซีเชลล์ (C Shell) หรือ csh ซึ่งพัฒนาโดยนายบิล จอย (Bill Joy) (หนึ่งในผู้ก่อตั้งบริษัทซันไมโครซิสเต็ม (Sun Microsystems) ซึ่งเป็นผู้ผลิตและจำหน่ายระบบคอมพิวเตอร์ที่ใช้ยูนิกซ์รายใหญ่ที่สุดรายหนึ่งในปัจจุบัน) ซึ่งในสมัยนั้นอยู่ที่มหาวิทยาลัยแคลิฟอร์เนีย (California) โปรแกรมนี้ถือเป็นส่วนหนึ่งของระบบยูนิกซ์รุ่นของเบิร์กลีย์ หรือที่เรียกกันว่าเบิร์กลีย์ซิสเต็มดิสทริบิวชัน (Berkeley System Distribution) หรือ BSD Unix สาเหตุที่เรียกเชลล์นี้ว่าซีเชลล์ (C Shell) เพราะมีรูปแบบของคำสั่งและการใช้งานที่คล้ายกับโปรแกรมภาษา C ทำให้ผู้ใช้ที่เป็นโปรแกรมเมอร์มีความคุ้นเคยกับการใช้งานเชลล์นี้มาก

โปรแกรมเชลล์ตัวอื่นที่ถูกพัฒนาขึ้นมาภายหลังและได้รับความนิยมสูงอีกโปรแกรมหนึ่งก็คือคอร์นเชลล์ (Korn Shell) หรือ ksh ถูกพัฒนาขึ้นมาโดยนายเดวิด คอร์น (David Korn) จากเอทีแอนด์ทีเบลแลบ (AT&T Bell Lab) ในราวกลางทศวรรษ 1980 คอร์นเชลล์นี้มีความเข้ากันได้กับเบอร์นเชลล์ได้โดยง่าย นอกจากนี้คอร์นเชลล์ก็ยังมีรูปแบบพิเศษบางอย่างที่ถูกเพิ่มเติมเข้ามาเพื่อให้ผู้ใช้สามารถใช้งานได้สะดวกขึ้นด้วย

ในระบบลินุกซ์สามารถเลือกติดตั้งเชลล์ได้หลายแบบ ซึ่งสามารถตรวจสอบได้ว่าขณะนี้กำลังใช้เชลล์อะไรอยู่โดยใช้คำสั่งต่อไปนี้

```
$ echo $SHELL
/bin/bash
```

ตัวอย่างนี้แสดงว่ากำลังใช้ bash หรือ Bourne again shell อยู่ หากต้องการเปลี่ยนหรือเรียกใช้งานเชลล์ชนิดอื่นๆ ขึ้นมา ก็สามารถใช้คำสั่งโดยป้อนชื่อของเชลล์ที่ต้องการลงไปได้เช่น

```
$ echo (call Bourne again shell)
$ csh (call c shell)
```

หรือสามารถเปลี่ยนแปลงดีฟอลต์เชลล์ (default shell) ที่ถูกเรียกขึ้นมาทุกครั้งที่มีการล็อกอินเข้าใช้งานระบบได้จากแฟ้มข้อมูล /etc/passwd

3.5.2 การเขียนโปรแกรมเชลล์

การเขียนโปรแกรมเชลล์หรือ เชลล์สคริปต์ (Shell script) เป็นการนำเอาคำสั่งต่างๆ ของระบบมารวมเข้าด้วยกัน โดยมีคำสั่งควบคุมการทำงาน (flow control) คอยเป็นตัวบังคับ และควบคุมรูปแบบของโปรแกรม

- การควบคุมลักษณะของการแสดงผล

การควบคุมการแสดงผลบนจอภาพใช้รหัสพิเศษที่ขึ้นต้นด้วยเอสเคป (escape) หรือ \033 เรียก รหัสชุดเหล่านี้ว่าเอสเคปซีควเนต์ (escape sequence) สามารถบังคับให้ระบบแสดงผลด้วยตัวอักษรแบบ กระพริบ,แบบกลับขาวเป็นดำ หรือขีดเส้นใต้ได้ ตัวอย่างของรหัสต่างๆ ที่ใช้มีดังนี้

\033[line;colH	ตำแหน่งของเคอร์เซอร์ไปอยู่บนบรรทัด line และคอลัมน์ col
\033[2J	ลบหน้าจอ (คล้ายกับคำสั่ง tput clear บนยูนิกซ์หรือคำสั่ง CLS บน DOS)
\033[4m	เข้าสู่โหมดขีดเส้นใต้ ตัวอักษรที่พิมพ์ต่อจากรหัสชุดนี้เป็นตัวอักษรที่มีการขีดเส้นใต้ทั้งหมด
\033[5m	เข้าสู่โหมดกระพริบ (blink)
\033[7m	โหมดกลับขาวดำ (reverse)
\033[m	กลับสู่โหมดปกติ

สามารถใช้คำสั่ง echo เพื่อส่งรหัสเหล่านี้ได้ สำหรับรหัส Esc ให้กดปุ่ม Ctrl-v (กดปุ่ม Ctrl ค้างไว้และตามด้วย v) หลังจากนั้น เมื่อกดปุ่มใดก็เป็นการส่งรหัสประจำปุ่มนั้นไปให้ระบบ ในที่นี้จะกดปุ่ม Esc เมื่อกดแล้วเห็นเป็นตัวอักษร “^[" หากต้องการกลับสู่โหมดตัวอักษรกระพริบ ต้องกดปุ่ม Ctrl-v ตามด้วยปุ่ม Esc ปุ่ม [,5 และ m ตามลำดับ

```
$ echo ^[[5m
```

เมื่อพิมพ์ตัวอักษรหลังจากนี้ เห็นเป็นตัวอักษรกระพริบทั้งหมด (สำหรับจอที่มีการกำหนดเป็น แอตทริบิวต์สี ก็จะเห็นเป็นสีที่แตกต่างจากปกติ) จากนั้นให้ป้อนคำสั่งต่อไปนี้เพื่อกลับสู่โหมดปกติ

```
$ echo ^[[m
```

ถ้าต้องการลบหน้าจอ ให้ใช้คำสั่ง tput clear คำสั่งต่างๆ สามารถนำมาใส่ไว้ในไฟล์เดียวกันได้ เรียกว่าเชลล์สคริปต์ และเรียกใช้งานโดยใช้คำสั่ง

```
$ bash myscript          (สำหรับ Bourne Again Shell)
```

หรือ

\$ ksh myscript (สำหรับ Korn shell)

หรือใช้คำสั่ง chmod เพื่อเปลี่ยนโหมดของไฟล์ให้เป็นชนิดที่ทำงานได้ (executable) แล้วเรียกเชลล์สคริปต์นั้นให้ทำงานด้วยการป้อนชื่อเข้าไปโดยตรง ดังนี้

\$ chmod +x myscript -> เปลี่ยนโหมดของไฟล์ให้เป็นแบบ executable
\$ myscript -> เรียกให้สคริปต์ทำงานโดยพิมพ์ชื่อลงไปโดยตรง

ตัวอย่างซอร์สโค้ดของสคริปต์

```
tput clear
echo  “^[[07;20H+-----+”
echo  “^[[07;38H^[[7m MENU  ^[[m”
echo  “^[[08;20H|”
echo  “^[[09;20H|    1.Option 1.    |”
echo  “^[[10;20H|    2.Option 2.    |”
echo  “^[[11;20H|    3.Option 3.    |”
echo  “^[[12;20H|    4.Option4.    |”
echo  “^[[13;20H|    5.Exit.        |”
echo  “^[[14;20H|”
echo  “^[[15;20H+-----+”
echo  “^[[16;20H|    Please select your choise :”
read  choise
```

- **การเปลี่ยนแปลงทิศทางของช่องทางการสื่อสาร (I/O redirection)**

ในการสื่อสารระหว่างเทอร์มินอลกับตัวโฮสต์ของคอมพิวเตอร์มีช่องทางที่ใช้กันอยู่ 3 ทาง คือ STDIN (ช่องทางรับข้อมูลมาตรฐาน), STDOUT (ช่องทางแสดงผลพื้มาตรฐาน) และ STDERR (ช่องทางแสดงผลพื้ที่เป็นข้อผิดพลาดมาตรฐาน) ทั้งนี้ความสามารถในเรื่องการจัดการเทอร์มินอลหรือเรื่องของการเปลี่ยนทิศทางของช่องทางการสื่อสาร (I/O redirection) ไม่ได้เป็นความสามารถจากโปรแกรมแอปพลิเคชันแต่เป็นความสามารถจากตัวระบบปฏิบัติการเอง

ลินุกซ์ได้รับอิทธิพลมาจากระบบยูนิกซ์ จึงยังคงความสามารถเกี่ยวกับเรื่องของการจัดการเทอร์มินอลและการเปลี่ยนทิศทางของช่องทางการสื่อสารไว้อย่างครบถ้วน จึงสามารถเปลี่ยนแปลงช่องทางการสื่อสารเหล่านี้จากเชลล์ได้ตลอดเวลา

- การเปลี่ยนแปลงช่องทางการแสดงผลพื้่ออก (standard output redirection)

ในที่นี้ยกตัวอย่างการใช้งานคำสั่ง “cat” ซึ่งรอรับข้อมูลจากคีย์บอร์ด แล้วแสดงผลพื้่อกลับมาทางจอภาพ ดังนั้นในที่นี้ STDIN คือคีย์บอร์ด และ STDOUT คือจอภาพหรือมอนิเตอร์

```
$ cat
This is first line          ->   พิมพ์ข้อความ “This is first line”
This is first line          ->   cat แสดงผลลัพธ์ คือ ข้อความที่เพิ่งพิมพ์ลงไป
This is second line         ->   พิมพ์ข้อความ “This is second line”
This is second line         ->   cat แสดงผลลัพธ์ คือ ข้อความที่เพิ่งพิมพ์ลงไป
```

ต่อไปนี้จะเปลี่ยนช่องทางการแสดงผลพื้ของโปรแกรม ซึ่งปกติแสดงออกทางจอภาพให้ไปเก็บลงไฟล์แทน

```
$ cat
This is first line          ->   พิมพ์ข้อความ “This is first line”
This is second line         ->   พิมพ์ข้อความ “This is second line”
```

โปรแกรม cat จัดเก็บผลลัพธ์ลงในไฟล์ที่ชื่อ “my file” แทน นั่นคือ STDOUT ได้ถูกเปลี่ยนเป็นไฟล์ “myfile” แล้ว เครื่องหมาย “>” ใช้ในการเปลี่ยนช่องทางการแสดงผลพื้ เป็นการเขียนทับไฟล์ที่ระบุชื่อ ถ้าต้องการให้เขียนผลลัพธ์ต่อท้ายข้อมูลเดิมที่มีอยู่แล้วในไฟล์ ต้องใช้เครื่องหมาย “>>” ดังตัวอย่าง

```
$ cat >> myfile
```

- การเปลี่ยนแปลงช่องทางการนำข้อมูลเข้า (standard input redirection)

ในที่นี้ ขอยกตัวอย่างคำสั่ง sort ซึ่งโดยปกติรับข้อมูลเข้าจากคีย์บอร์ดแล้วแสดงผลพื้่อกลับออกมาทางจอภาพ ดังนั้น STDIN คือคีย์บอร์ด และ STDOUT คือจอภาพ
การทำงานของคำสั่ง sort

```
$ sort
Peter          ->   พิมพ์ Peter
Mary           ->   พิมพ์ Mary
John           ->   พิมพ์ John เสร็จแล้วกด Ctrl-D
John           ->   sort จัดเรียงข้อมูลใหม่
Mary
```


Peter

เตรียมไฟล์ชื่อ “namelist” ไว้ให้กับโปรแกรม sort

\$ cat > namelist

Peter	->	พิมพ์ Peter
Mary	->	พิมพ์ Mary
John	->	พิมพ์ John เสร็จแล้วกด Ctrl-D

การเปลี่ยนทิศทางของช่องทางนำข้อมูลเข้าจากคีย์บอร์ด ไปเป็นการนำข้อมูลเข้าจากไฟล์ที่ชื่อ “namelist” ทำดังต่อไปนี้

```
$ sort < namelist
Peter
Mary
John
```

กล่าวคือ เครื่องหมายที่ใช้ในการเปลี่ยนช่องทางนำข้อมูลเข้าคือ “<” จากตัวอย่างข้างต้น STDIN เปลี่ยนไปเป็นไฟล์ที่ชื่อ “namelist”

- การเปลี่ยนทั้ง STDIN และ STDOUT พร้อมกัน

ยกตัวอย่าง เช่น

```
$ sort < namelist > sortlist
```

คำสั่งนี้โปรแกรม sort รับข้อมูลจากไฟล์ “namelist” และเขียนผลลัพธ์ไปไว้ในไฟล์ “sortlist” สามารถลองใช้คำสั่ง cat ตรวจสอบข้อมูลในไฟล์ “sortlist” ได้

- การใช้งานไปป์ (pipe)

การใช้งานไปป์ (pipe) เป็นการนำเอาผลลัพธ์จากคำสั่งหนึ่งไปเป็นข้อมูลเข้าของอีกคำสั่งหนึ่ง ซึ่งโปรแกรมที่มีลักษณะการใช้งานเป็นแบบฟิลเตอร์ ได้แก่ cat, sort, more, grep, tee และ tr

ตัวอย่างการใช้ไปป์

```
$ ls | grep file | sort
```

เป็นการเรียงลำดับข้อมูลที่ได้จากการคัดเลือกรายชื่อไฟล์ที่มีชื่อ “file” เป็นส่วนประกอบ

- คำสั่งควบคุมทิศทางแบบวนรอบ (loop flow control)

คำสั่งควบคุมทิศทางแบบวนรอบหรือวนลูป มีอยู่ 3 แบบคือ for, while และ until

for loop

โครงสร้าง

<pre>For VAR in Arg-list do command list done</pre>

คำสั่ง for นี้ ทำงานวนรอบ จนกระทั่งสมาชิกใน “Arg-list” (หมายถึง argument list) ถูกใช้หมด ชุดคำสั่งในลูป for ต้องอยู่ภายใต้คำบังคับ “do” และ “done” การใช้ for ส่วนใหญ่ ใช้กับลูปที่รู้จำนวนรอบที่ต้องทำและ “Arg-list” แน่แน่นอน

ตัวอย่าง

```
SUM=0
for i in 1 2 3 4 5 6 7 8 9
do
    SUM='expr $SUM + $ i'
done
echo "Sum is $SUM"
```

ให้บันทึกเซลล์สคริปต์ข้างต้นเป็นไฟล์ชื่อ ex8 แล้วสั่งให้โปรแกรมทำงานดังนี้

```
$ ex8
Sum is 45
```

จากตัวอย่างเป็นการรวมตัวเลขจำนวนเต็มตั้งแต่ 1 ถึง 9 เสร็จแล้วพิมพ์ค่าที่ได้

while และ until

โครงสร้างของ while

<pre>while [true condition] do command list done</pre>
--

คำสั่ง while ใช้กับเงื่อนไขที่เป็นจริง ตัวอย่าง เช่น

```

SUM=0
i = 1
while [ $i -lt 10 ]
do
    SUM='expr $SUM + $i'
    i='expr $i + 1'
done
echo "Sum is $SUM"

```

โปรแกรมตัวอย่างข้างบนทำงานเหมือนกับ ex8 แต่ในที่นี้ใช้รูปแบบของ while loop โดยโปรแกรมบวกตัวเลข “i” ไปเรื่อยๆ เริ่มตั้งแต่ 1 และเพิ่มค่า i ขึ้นทีละหนึ่ง รูปนี้ทำงานไปเรื่อยๆ ตราบใดที่ค่า i ยังน้อยกว่า 10 และเมื่อค่า i มากกว่าเท่ากับ 10 แล้ว ออกจากลูปและพิมพ์ผลบวกที่ได้ทั้งหมด

โครงสร้างของ until

<pre> until [false condition] do <i>command list</i> done </pre>

คำสั่ง until ใช้กับเงื่อนไขที่เป็นเท็จ ตัวอย่างเช่น

```

SUM=0
i=1
until [ $i -eq 10 ]
do
    SUM='expr $SUM + $i'
    i='exp $i +1'
done
echo "Sum is $Sum"

```

ตัวอย่างข้างต้นนี้ก็เป็นโปรแกรมที่ให้ผลลัพธ์แบบเดียวกับในตัวอย่างที่ใช้โครงสร้างของ for และ while นั่นเอง และจะเห็นว่าลักษณะคล้ายกับโปรแกรมที่ใช้โครงสร้างแบบ while-loop มาก เพียงแต่เงื่อนไขที่ใช้ตรวจสอบกลับกันเท่านั้น กล่าวคือเงื่อนไขนี้ให้ตรวจสอบว่าค่าของตัวแปร i เท่ากับ 10 หรือยัง ทั้งนี้ until-loop จะทำการวนลูปทำงานไปเรื่อยๆ ตราบใดที่เงื่อนไขที่ตามหลัง until มานั้น ยังคงเป็นเท็จ

ในการเขียนโปรแกรมสามารถเลือกใช้เฉพาะ while-loop หรือ until-loop ตัวใดตัวหนึ่งเพียงอย่างเดียวก็ได้ เพราะเงื่อนไขของลูปทั้งสองแบบสามารถดัดแปลงให้ทดแทนกันได้ ดังที่ได้แสดงในโปรแกรมตัวอย่าง

Break และ continue

คำสั่ง break และ continue จะหยุดการทำงานของชุดคำสั่งที่ตามหลังสองคำสั่งนี้ สำหรับคำสั่ง break เมื่อหยุดแล้วก็กระโดดออกไปนอกลูป ตรงจุดที่อยู่ถัดจากคำสั่ง “done” ส่วน continue กลับไปทำงานในลูปต่อตรงจุดต้นลูปที่อยู่ถัดจากคำสั่ง “do” ดังตัวอย่างต่อไปนี้

```
SUM=0
While [true]
do
    echo -n "Please enter number 1-9, except 5, q=quit)"
    read INPUT
    if [ "$INPUT" = "q" ]
    then
        echo "Break loop now!"
        break
    elif [ $INPUT -eq 5 ]
    then
        echo "Please not input 5"
        continue
    elif [ $INPUT -gt 9 ]
    then
        echo "Please do not input data > ?"
        continue
    fi
    SUM=`expr $SUM + $INPUT`
done
echo "Sum is $SUM"
```

ในตัวอย่างนี้นอกจากแสดงถึงการใช้ break กับ continue แล้ว ยังแสดงให้เห็นถึงการใช้ while ในแบบที่นิยมใช้กันอีกลักษณะหนึ่ง คือการใช้กับเงื่อนไข “true” วิธีการใช้งานแบบนี้ทำให้โปรแกรมวนลูปแบบไม่รู้จบ ซึ่งมักใช้กับการวนทำงานเพื่อรับข้อมูลเข้าไปตามตัวอย่างข้างต้น โดยมีเงื่อนไขของการรับข้อมูลบางรูปแบบที่ใช้กระโดดออกจากลูปเพื่อจบการทำงาน ซึ่งในที่นี้ก็ใช้คำสั่ง “break” เมื่อผู้ใช้งานป้อนตัวอักษร “q” เข้ามา ทั้งนี้อาจใช้คำสั่ง “exit” แทน “break” ก็ได้ แต่ทำให้ไม่มีการทำงานตามคำสั่งในส่วนที่พิมพ์ผลลัพธ์ ซึ่งอยู่ท้ายโปรแกรมไปด้วย

สำหรับคำสั่ง “continue” ใช้กับการป้อนข้อมูลที่เป็นตัวเลข 5 ซึ่งในที่นี้เป็นการข้ามการบวกเลข 5 ไป ตัวเลขที่ใส่ได้และโปรแกรมทำการบวกเลขให้มีเฉพาะตัวเลข 1-4 และ 6-9 เท่านั้น ส่วนตัวเลขที่น้อยกว่า 1 และมากกว่า 9 ก็จะถูกคำสั่ง “continue” ข้ามส่วนของโปรแกรมที่เป็นการบวกเลขไปเช่นเดียวกัน

คำสั่ง `continue` และ `break` นี้ กระโดดออกไปจากลูปชั้นที่โปรแกรมกำลังทำงานอยู่ แต่สามารถระบุให้ `continue` และ `break` กระโดดออกไปยังลูปที่ชั้นใดก็ได้ (ในกรณีที่มีลูปมีการซ้อนอยู่หลายชั้น) วิธีการใช้งานก็เพียงแค่เติมตัวเลขตามท้ายคำสั่ง `break` และ `continue` เท่านั้น ตัวอย่างเช่น “`break2`” ทำการกระโดดออกจากลูปไปสองชั้น (ในอยู่หลัง `done` ของลูปที่สอง)

3.5.3 ตัวแปรเชลล์

ตัวแปรเชลล์ (shell variable) มีการเก็บค่าไว้ในลักษณะของข้อความหรือสตริง (string) ในกรณีที่ใช้งานเป็นตัวเลขก็ต้องใช้โปรแกรม `expr` เข้ามาช่วยแปลงข้อมูลก่อน นอกจากนี้การกำหนดค่าให้กับตัวแปร ถือว่าเป็นการประกาศตัวแปรนั้นขึ้นมาด้วย ซึ่งมีรูปแบบดังต่อไปนี้

ชื่อตัวแปร=ค่าตัวแปร

โดยห้ามมีช่องว่าง (space) คั่นระหว่างเครื่องหมายเท่ากับ ตัวอย่างเช่น

NAME=KLAUSE

หากมีการกำหนดค่าที่เป็นประโยคข้อความยาวๆ ที่มีช่องว่างอยู่ในประโยคนั้น ต้องใช้เครื่องหมายอัญประกาศ (“ ”) ครอบประโยคนั้นเช่น

NAME="Peter Klaus"

การตั้งชื่อของตัวแปรสามารถใช้ตัวอักษร ตัวเลข ตัวขีดเส้นใต้ (underscore) แต่ห้ามใช้ตัวอักษรพิเศษต่างๆ เช่น `$`, `*`, `#`, `(` และห้ามขึ้นต้นด้วยตัวเลข ตัวอย่างชื่อตัวแปรเช่น `VARI`, `VAR_NAME`, `Variable` แต่โดยทั่วไปแล้วการตั้งชื่อของตัวแปรมักใช้อักษรตัวใหญ่ในการตั้งชื่อ ส่วนการใช้งานตัวแปรเชลล์หรือการอ้างถึงค่าของตัวแปร ต้องใช้เครื่องหมาย “`$`” นำหน้าตัวแปรนั้นเช่น ถ้าใช้คำสั่ง `$echo $NAME`

ในกรณีที่กำหนดค่าของตัวแปรเป็น “Peter Klaus” ได้ผลลัพธ์เป็น Peter Klaus แต่ถ้าใช้คำสั่ง `$echo NAME` เนื่องจากไม่ได้อ้างถึงค่าของตัวแปร ดังนั้นได้ผลลัพธ์เป็น

NAME

ในกรณีต้องการให้เชลล์สามารถแยกชื่อของตัวแปรออกจากคำอื่นๆ สามารถทำได้โดยครอบชื่อของตัวแปรนั้นด้วยเครื่องหมายปีกกา “`{ }`” ดังตัวอย่าง

```
$ MYDIR=/home/mary/
$ cat ${MYDIR}myfile
```

คำสั่งข้างต้นเป็นการสั่งให้พิมพ์รายละเอียดของแฟ้มที่ชื่อ `/home/mary/myfile` (ซึ่งเป็นการเอาค่าของตัวแปร(`home/mary/`) มาต่อกับชื่อไฟล์(`myfile`)) ออกมาให้ หากไม่ครอบชื่อของตัวแปรด้วยเครื่องหมายปีกกาแล้ว เชลล์จะมองชื่อตัวแปรเป็น `$MYDIRmyfile` (ซึ่งไม่มีค่าอะไรอยู่)

การยกเลิกตัวแปรสามารถทำได้โดยใช้คำสั่ง

\$ unset ชื่อตัวแปร

- ขอบเขตของตัวแปรเชลล์

ตัวแปรเชลล์ที่ถูกกำหนดขึ้นมามีลักษณะเป็น ตัวแปรท้องถิ่น (local variable) คือรู้จักเฉพาะในเชลล์ของชั้นตนเองเท่านั้น เชลล์ในชั้นอื่นๆ ที่ถูกเรียกซ้อนกันจะไม่รู้จักตัวแปรดังกล่าว จนกว่าจะประกาศตัวแปรนี้ให้เชลล์ชั้นอื่นๆ ได้รู้จักก่อน ซึ่งทำได้โดยใช้คำสั่ง “export”

การเรียกเชลล์ซ้อนกันหลายชั้นหรือ “ซับเชลล์” (sub shell) หมายถึงการพิมพ์คำสั่ง “sh” หรือ “ksh” ซึ่งเป็นการเรียกเบอร์นเชลล์หรือคอร์นเชลล์ซ้อนขึ้นมาบนเชลล์ปัจจุบัน เมื่อเรียกขึ้นมาแล้วจะเห็นเป็นเครื่องหมาย prompt ของระบบตามปกติ ซึ่งหมายความว่าซับเชลล์ได้ถูกเรียกขึ้นมาใหม่แล้ว อาจจะใช้คำสั่ง “ps” เพื่อตรวจสอบว่าซับเชลล์ถูกเรียกขึ้นมาหรือไม่ ดังนี้

```
$ ps
PID    TTY    STAT   TIME       COMMAND
962    p2     S       0:00      login -h localhost -p
963    p2     S       0:00      -bash
974    p2     R       0:00      ps
```

ตัวอย่างเชลล์แรกที่ลินุกซ์กำหนดไว้ให้ เมื่อทำการล็อกอินเข้ามาใช้ระบบครั้งแรกคือ bourne again shell (bash) ซึ่งเป็นโปรเซสหมายเลข 963

```
$ sh
$ ps
PID    TTY    STAT   TIME       COMMAND
962    p2     S       0:00      login -h localhost -p
963    p2     S       0:00      -bash
975    p2     S       0:00      sh
976    p2     R       0:00      ps
```

เมื่อเรียกซับเชลล์เห็นโปรเซสหมายเลข 975 ซึ่งเป็นโปรเซสของเบอร์นเชลล์ที่เรียกด้วยคำสั่ง “sh” ขึ้นมา

```
$ exit
exit
$ ps
PID    TTY    STAT   TIME       COMMAND
```

```

962  p2    S    0:00    login -h localhost -p
963  p2    S    0:00    -bash
989  p2    R    0:00    ps

```

สามารถออกจากซันเชลล์ได้โดยการพิมพ์คำสั่ง “exit” ตามปกติ เมื่อใช้คำสั่ง exit เพื่อออกจากซันเชลล์ จะเห็นว่าโปรเซสของเบอร์นเชลล์หายไป เนื่องจากถูกฆ่าด้วยคำสั่ง exit ไปแล้ว แสดงว่าตอนนี้ได้กลับมาอยู่ที่เชลล์แรกซึ่งก็คือ bash อีกครั้ง (โปรเซสหมายเลข 963)

ตัวอย่างการเรียกซันเชลล์และการประกาศตัวแปรให้เชลล์ชั้นอื่น ๆ ได้รู้จัก

```

$ NAME=KLAUSE          -> ประกาศตัวแปร NAME ในเชลล์ชั้นปัจจุบัน
$ sh                   -> เรียกซันเชลล์ซ้อนขึ้นมา
$ echo $NAME           -> ไม่พิมพ์ผลลัพธ์ใดๆ ออกมา เนื่องจากซันเชลล์ไม่รู้จัก
$ exit                 -> ออกจากซันเชลล์กลับมาสู่เชลล์ปัจจุบัน
$ export NAME          -> ประกาศตัวแปร NAME ให้กับเชลล์ชั้นอื่นๆ
$ sh                   -> เรียกซันเชลล์อีกครั้งหนึ่ง
$ echo $NAME
KLAUSE                 -> เมื่อซันเชลล์รับรู้ตัวแปรแล้ว จึงพิมพ์ผลลัพธ์ออกมาให้
$ readonly NAME        -> คำสั่งนี้ทำให้ไม่สามารถเปลี่ยนแปลงค่าตัวแปรได้

```

- **ตัวแปรสวอนหรือตัวแปรระบบ**

ในระบบลินุกซ์และยูนิกซ์มีตัวแปรสวอน (reserved) ที่ตัวระบบปฏิบัติการหรือโอเอส(OS) นำไปใช้ประโยชน์โดยเฉพาะเท่านั้น แต่หากรู้ความหมายของตัวแปรเหล่านั้นก็สามารถแก้ไขเปลี่ยนแปลงค่าของตัวแปรดังกล่าวได้ สามารถตรวจสอบค่าของตัวแปรระบบได้โดยใช้คำสั่งนี้

```

$ set
PPID=373
PS1=$
PS2=>
PS4=+
PWD=/home/twg
SHELL=/bin/bash
SHLVL=1
TERM=xterm
UID=406

```

```

USER=twg
WINDOWID=37748749
_=set
i=etc/profile.d/mh.sh

```

พบว่าระบบแสดงรายละเอียดของชื่อและค่าของตัวแปรสวณต่างๆ ออกมาให้ ต่อไปเป็นคำอธิบายของตัวแปรสวณบางตัว

\$ HOME	->	Home directory
\$ PATH	->	search patch เมื่อคำสั่งเรียกโปรแกรมให้ทำงานระบบทำการค้นหาตัวโปรแกรมนั้นจากรายชื่อของไดเรกทอรีที่ได้ระบุไว้ในตัวแปรนี้
\$ MAIL	->	mailbox
\$ PS1	->	prompt ตัวที่ 1 ของเชลล์
\$ PS2	->	prompt ตัวที่ 2 ของเชลล์
\$ LOGNAME	->	ชื่อทะเบียนผู้ใช้ระบบ (login name)
\$ TERM	->	ชนิดของจอ

นอกจากการใช้คำสั่ง “set” เพื่อแสดงรายชื่อและค่าของตัวแปรต่างๆ ออกมาแล้ว ยังสามารถใช้คำสั่ง “echo <ชื่อตัวแปร >” เพื่อแสดงค่าของตัวแปรดังกล่าวออกมาก็ได้เช่นกัน และการเปลี่ยนหรือกำหนดค่าให้กับตัวแปรก็สามารถทำได้เช่นเดียวกับตัวแปรเชลล์ปกติทั่วไป ดังตัวอย่าง

\$ echo \$PS1	->	ให้แสดงค่าของตัวแปร PS1 (prompt ของระบบ)
\$	->	ระบบพิมพ์ตัวอักษร “\$” ซึ่งถูกใช้เป็น prompt ออกมาให้
\$ PS1=”Linux:> “	->	กำหนดค่าของ prompt ใหม่เป็น “Linux:>”
Linux:>	->	prompt ของระบบเปลี่ยนเป็น “Linux:>”

3.5.4 การสร้างเชลล์สคริปต์

การสร้างเชลล์สคริปต์ เป็นการนำเอาคำสั่งต่างๆ ในลินุกซ์มาประกอบเข้าเป็นไฟล์ จากคำสั่งต่อไปนี้ ลองสร้างไฟล์ชื่อ “ex1”

\$ sh	#call subshell
\$ echo “Hello world”	#print “Hello world”
\$ date	#print date information
\$ pwd	#print current directory
\$ exit	#exit from subshell

เมื่อบันทึกข้อมูลเก็บลงไฟล์ชื่อ “ex1” แล้ว สามารถสั่งให้โปรแกรมทำงานได้สองวิธี คือ

- 1) สั่งผ่านเชลล์ ให้พิมพ์คำสั่งดังนี้

```
$ sh ex1
```

วิธีการสั่งงานโปรแกรมแบบนี้ “ex1” ต้องถูกตั้งไว้ว่าอนุญาตให้สามารถอ่านไฟล์ได้ (readable) ไม่เช่นนั้นเชลล์จะไม่สามารถทำให้โปรแกรมทำงานได้

- 2) สั่งโดยตรงจากชื่อโปรแกรม ให้พิมพ์คำสั่ง

```
$ ex1
```

สำหรับวิธีนี้โปรแกรม “ex1” ต้องถูกตั้งว่าอนุญาตให้สามารถสั่งทำงานได้ (executable) ไม่เช่นนั้น เชลล์จะปฏิเสธการสั่งงานโปรแกรม (Permission denied)

สามารถกำหนดคุณสมบัติให้ไฟล์อนุญาตให้มีการสั่งทำงานได้โดยใช้คำสั่งดังนี้

```
$ chmod u+x ex1      อนุญาตให้สั่งทำงานได้ในระดับผู้ใช้
$ chmod g+x ex1      อนุญาตให้สั่งทำงานได้ในระดับกลุ่ม
$ chmod o+x ex1      อนุญาตให้สั่งทำงานได้ในระดับบุคคลอื่นๆ
$ chmod +x ex1 และ $ chmod a+x ex1    เปิดให้สั่งทำงานได้ในทุกระดับชั้น
```

หากกำหนดให้ไฟล์อนุญาตให้มีการสั่งทำงานได้แล้ว และพิมพ์ชื่อโปรแกรมเพื่อสั่งให้ทำงาน แต่ปรากฏว่าเชลล์ยังคงปฏิเสธการสั่งงาน (เช่นตอบว่า ex1 : command not found) แสดงว่าเชลล์ไม่ทราบว่าจะไปค้นหาชื่อโปรแกรมได้จากไครเทกทอรีใด วิธีการระบุชื่อที่อ้างอิงกับไครเทกทอรีก็ทำได้ 2 วิธีคือ

- 1) Relative Pathname วิธีระบุโดยอ้างอิงจากไครเทกทอรีปัจจุบัน

```
$ ./ex1
```

- 2) Full Pathname ระบุแบบอ้างอิงชื่อเต็ม เช่นสมมติว่าไฟล์อยู่ในไครเทกทอรี /home/user1

```
$ /home/user1/ex1
```

การเรียกใช้งานโปรแกรมโดยที่ต้องระบุชื่อไครเทกทอรีด้วย ก่อนข้างเป็นเรื่องยุ่งยาก หากเก็บโปรแกรมไว้ในไครเทกทอรีเฉพาะ เช่น /home/user1/shell สำหรับเก็บโปรแกรมเชลล์ที่เรียกใช้บ่อยๆ ก็ทำให้สะดวกขึ้น ในระบบลินุกซ์มีวิธีการกำหนดให้เชลล์ทำการค้นหาโปรแกรมที่สามารถสั่งให้ทำงานได้จากไครเทกทอรีกลุ่มหนึ่ง ตามที่ระบุไว้ในตัวแปรของระบบชื่อ PATH (หรือที่เรียกว่า search path) สามารถตรวจสอบชื่อไครเทกทอรีที่เชลล์ค้นหาเมื่อมีการเรียกใช้งานโปรแกรมโดยไม่ระบุชื่อไครเทกทอรีได้ โดยการใช้คำสั่ง

```
$ echo $PATH
```

```
/usr/local/bin:/bin:/user/bin:/usr/X11R6/bin
```

จากผลลัพธ์ข้างต้น หากมีการสั่งงานโปรแกรมโดยไม่ระบุชื่อไครเทกทอรีดังกรณีข้างต้น เชลล์ค้นหาโปรแกรมจากไครเทกทอรีทั้งสี่ที่ระบุไว้ในตัวแปร PATH คือ /usr/local/bin , /bin, /usr/bin และ /usr/X11R6/bin ซึ่งแต่ละไครเทกทอรีเหล่านี้ไว้สำหรับค้นหาและเรียกโปรแกรมที่ใช้บ่อยๆ (เช่น “ls” อยู่

ในไดเรกทอรี “/bin” และ “/usr/bin” อยู่ในไดเรกทอรี “/user/bin” เป็นต้น) สำหรับโปรแกรม ex1 นั้น เนื่องจากอยู่ในไดเรกทอรี /home/user1 ดังนั้นเชลล์จึงไม่สามารถค้นหาพบได้ จึงต้องเพิ่มไดเรกทอรีนี้ลงไปในตัวแปร PATH เอง

วิธีการเพิ่มไดเรกทอรีใช้คำสั่งดังต่อไปนี้

```
$ PATH=$ PATH:/home/user1
```

```
$ export PATH
```

สำหรับคำสั่งแรกเป็นการนำเอาชื่อของกลุ่มไดเรกทอรีเดิม (\$PATH) มาต่อเข้ากับไดเรกทอรีใหม่ (/home/user) หากไม่ระบุชื่อไดเรกทอรีเดิม (\$PATH) กลายเป็นการแทนที่ของเดิมทั้งหมดด้วยของใหม่ไป ซึ่งถ้าเป็นเช่นนั้นทำให้ไม่สามารถเรียกใช้โปรแกรมที่เคยใช้ได้อีก (เช่น โปรแกรม ls) นอกจากนี้ หากต้องการให้มีการสั่งงานโปรแกรมที่อยู่ในไดเรกทอรีปัจจุบัน (current directory) ได้ด้วย ต้องใช้คำสั่งต่อไปนี้

```
$ PATH=$ PATH
```

```
$ export PATH
```

คำสั่งข้างต้นเป็นการเพิ่มไดเรกทอรีปัจจุบัน (.) ลงไปในตัวแปร PATH ด้วย คือจุด (dot) “.” ซึ่งเป็นการแทนความหมายของไดเรกทอรีปัจจุบัน ให้ใช้คำสั่ง “echo” เพื่อตรวจสอบว่าได้เพิ่มไดเรกทอรีเข้าไปในตัวแปร PATH เรียบร้อยหรือไม่ ดังนี้

```
$ echo $PATH
```

```
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin:/home/user1:.
```

สำหรับการกำหนดให้เพิ่มไดเรกทอรีปัจจุบัน (.) ลงไปในตัวแปร PATH นี้ ถ้าคำนึงถึงเรื่องของการรักษาความปลอดภัยแล้วเป็นเรื่องที่ไม่ควรทำอย่างยิ่ง ปกติระบบยูนิกซ์ส่วนใหญ่ไม่นิยมให้ผู้ใช้กลุ่มที่อยู่ในระดับของการดูแลรักษาระบบและมีสิทธิ์สูงที่สุดในระบบอย่าง root มีไดเรกทอรีปัจจุบัน (.) อยู่ในตัวแปร PATH เลยเพราะหากกำหนดให้ไดเรกทอรีปัจจุบันอยู่ในตัวแปร PATH แล้ว อาจมีการอัปเดตโปรแกรมหรือสคริปต์ที่มีจุดประสงค์ในการบุกรุกระบบไว้ในไดเรกทอรีปัจจุบัน และผู้บุกรุกที่เข้าใช้ระบบก็สามารถเรียกโปรแกรมให้ทำงานได้ทันที โดยไม่จำเป็นต้องระบุไดเรกทอรีปัจจุบัน และผู้บุกรุกที่เข้าใช้ระบบก็สามารถเรียกโปรแกรมให้ทำงานได้ทันที โดยไม่จำเป็นต้องระบุไดเรกทอรีเต็มรูปแบบกำกับไปด้วยวิธีการกำหนดตัวแปร PATH แบบนี้ช่วยป้องกันการบุกรุกระบบได้ในระดับหนึ่ง

โดยปกติแล้วเมื่อเรียกเชลล์สคริปต์ให้เริ่มทำงาน หมายถึงการเริ่มต้นทำคำสั่งในสคริปต์ทีละคำสั่งตามลำดับ โดยก่อนการทำคำสั่งแรกในสคริปต์จะมีการเรียกซับเชลล์ให้ด้วย และหลังจากทำคำสั่งทุกคำสั่งในเชลล์สคริปต์เสร็จสิ้นแล้วก็เสมือนกับการออกจากซับเชลล์นั้นด้วยคำสั่ง “exit”

แม้ว่าไม่ได้ใส่คำสั่ง exit ไว้ในเชลล์สคริปต์ แต่ก็เสมือนว่ามีคำสั่งนั้นอยู่ท้ายเชลล์สคริปต์ด้วย และโดยทั่วไปหากเป็นการจบการทำงานแบบปกติที่ไม่มีข้อผิดพลาดเกิดขึ้น มีการส่งรหัสค่า “0” (return code 0) ออกมาให้ (คำสั่ง exit ที่ไม่มีตัวเลขตามท้ายนี้ถือว่าส่ง return code 0) แต่ถ้าเป็นการจบการ

ทำงานแบบไม่ปกติ คืออาจมีข้อผิดพลาดเกิดขึ้น ก็ส่งรหัสในค่าที่ไม่เป็น “0” ออกมาให้ (ปกติเป็นรหัส “1”) สำหรับรหัสค่าศูนย์ดังกล่าวนี้ นำไปใช้ทดสอบเงื่อนไขร่วมกันกับคำสั่งอื่นๆ ในเชลล์ต่อไป

ที่จริงไม่ได้มีการใส่ทั้งคำสั่งในการเรียกชัซเชลล์และคำสั่งที่ให้ออกจากชัซเชลล์นั้นลงไป ในเชลล์สคริปต์ด้วย แต่เมื่อสั่งให้เชลล์สคริปต์ทำงานแล้ว โปรแกรมเชลล์จะเติมคำสั่งทั้งสองลงไปให้โดยอัตโนมัติ นั่นหมายความว่าโดยปกติแล้วทุกคำสั่งในเชลล์สคริปต์ต้องทำงานภายใต้ชัซเชลล์ ซึ่งการทำงานภายใต้ชัซเชลล์นี้ก็มีข้อจำกัดดังที่อธิบายแล้ว นั่นคือภายในชัซเชลล์ไม่สามารถเรียกใช้งานตัวแปรจากเชลล์ชั้นบนก่อนหน้านั้นทั้งหมดได้ หากไม่ได้ทำการเอ็กซ์พอร์ตตัวแปรนั้นๆ เสียก่อน อย่างไรก็ตามก็มีวิธีที่บังคับให้เชลล์สคริปต์ทำงานในเชลล์ชั้นปัจจุบันได้โดยไม่ไปเรียกชัซเชลล์ ดังจะได้อธิบายต่อไป

3.5.5 วิธีการสั่งให้เชลล์สคริปต์ทำงาน

รูปแบบการสั่งงานเชลล์สคริปต์ มีได้ 4 แบบดังต่อไปนี้

แบบที่ 1

\$ script-name สคริปต์ต้องการ execute permission และวิ่งที่ subshell

แบบที่ 2

\$ sh script-name สคริปต์ต้องการ read permission และวิ่งที่ subshell

แบบที่ 3

\$.script-name สคริปต์วิ่งที่ current shell

แบบที่ 4

\$ exec scriptname สคริปต์วิ่งที่ current shell โดยทับเชลล์ชั้นปัจจุบันที่ทำงานอยู่เดิม

สำหรับการสั่งงานเชลล์สคริปต์ในแบบที่ 1 และแบบที่ 2 เมื่อล็อกอินเข้ามา ได้เชลล์ใช้งาน (ซึ่งปกติลินุกซ์เป็น bash) ในที่นี้เรียกว่าเชลล์เริ่มต้น (Parent Shell) เมื่อมีการสั่งให้เชลล์สคริปต์ทำงานตามรูปแบบที่ 1 หรือ 2 ก็เปรียบเสมือนมีการเรียกชัซเชลล์เกิดขึ้น (คำสั่ง sh) เมื่อเรียกชัซเชลล์แล้วก็ทำงานตามคำสั่งในเชลล์สคริปต์ทีละคำสั่งจนจบ ก็กลับไปสู่เชลล์เริ่มต้น (คำสั่ง exit) อีกทีหนึ่ง ในกรณีนี้หากเชลล์สคริปต์ต้องการใช้งานตัวแปรที่อยู่ในเชลล์เริ่มต้น ต้องมีการเอ็กซ์พอร์ตตัวแปรนั้นๆ ออกมาจากเชลล์เริ่มต้นเสียก่อน

สำหรับการสั่งงานในแบบที่ 3 มีผลให้เชลล์สคริปต์ทำงานในเชลล์ชั้นปัจจุบันทันที (ในที่นี้คือเชลล์เริ่มต้น) และทำงานตามคำสั่งในเชลล์สคริปต์ทีละคำสั่งจนจบ โดยไม่มีการเรียกชัซเชลล์ขึ้นมา ซึ่งทำให้สามารถใช้งานตัวแปรเชลล์ในเชลล์เริ่มต้นได้ทันที โดยไม่ต้องเอ็กซ์พอร์ตตัวแปรออกมาก่อน

สำหรับการสั่งงานในแบบที่ 4 เชลล์เริ่มต้นถูกทำลายไป แล้วนำเอาเชลล์สคริปต์มาแทนที่โพเชสของ Parent Shell ที่ถูกทำลาย ดังนั้น เมื่อเชลล์สคริปต์ทำงานทีละคำสั่งจนจบแล้วก็กลับไปสู่หน้าจอของการล็อกอินโดยอัตโนมัติ